

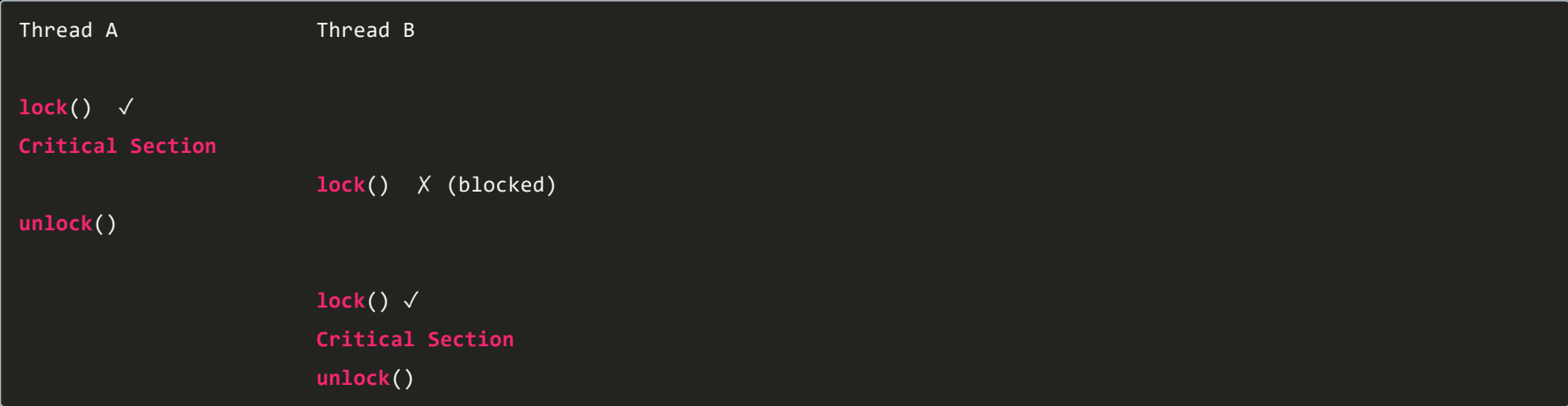
Synchronization Interview Notes (Intermediate)

1. Mutex (Mutual Exclusion) ★ ★ ★

Definition

A **mutex (Mutual Exclusion)** is a synchronization primitive used to ensure that **only one thread can access a critical section at a time**. It enforces **ownership**, meaning **only the thread that acquires (locks) the mutex can release (unlock) it**. Its primary purpose is to prevent **race conditions** while accessing shared resources.

Working



When a thread calls `lock()`:

- If the mutex is free → it acquires the lock and enters the critical section.
- If already locked → the thread blocks (sleeps) until the mutex is released.

Example

```
std::mutex mtx;
int counter = 0;

void increment() {
    mtx.lock();

    counter++;

    mtx.unlock();
}
```

Without the mutex, multiple threads may execute `counter++` simultaneously, causing a race condition.

Characteristics

- Provides **mutual exclusion**
- Has **ownership**
- Only the owner can unlock it
- Used to protect **shared data**
- Causes blocked threads to **sleep**, avoiding busy waiting

Advantages

- Prevents race conditions
- Efficient for long critical sections
- Simple and widely used

Disadvantages

- Context-switch overhead if the lock is unavailable
- Incorrect locking order may lead to deadlock

Interview Answer

A mutex is a synchronization primitive that provides mutual exclusion by allowing only one thread to execute a critical section at a time. It enforces ownership, so only the thread that locks the mutex can unlock it. It is primarily used to protect shared resources from race conditions.

2. Semaphore ★★ ★

Definition

A **semaphore** is a synchronization primitive represented by an integer counter used to **control access to shared resources** or **coordinate execution between threads**.

Unlike a mutex, a semaphore **does not enforce ownership**.

Semaphore Operations

wait() (P operation)

Attempts to acquire the semaphore.

```
count--  
  
if count < 0  
    block the thread
```

If no resources are available, the thread waits.

signal() (V operation)

Releases the semaphore.

```
count++  
  
if threads are waiting  
    wake one thread
```

Example

```
Semaphore = 2  
  
Thread A → wait() → enters  
Thread B → wait() → enters  
Thread C → wait() → blocks  
  
Thread A finishes  
signal()  
  
Thread C wakes up
```

Characteristics

- Integer counter
- Supports synchronization and resource management
- No ownership
- Can allow one or multiple threads simultaneously

Applications

- Producer–Consumer
- Resource pools
- Reader–Writer synchronization
- Thread signaling

Interview Answer

A semaphore is an integer-based synchronization primitive used to coordinate threads or control access to shared resources. Threads perform `wait()` to acquire the semaphore and `signal()` to release it. Unlike a mutex, semaphores do not enforce ownership.

3. Difference: Mutex vs Semaphore ☆☆☆

Feature	Mutex	Semaphore	Purpose	Mutual exclusion	Synchronization and resource management	Ownership	Yes	No	Unlock	Only owner
thread	Any thread can signal	Value	Locked / Unlocked	Integer counter	Multiple resources	No	Yes (counting semaphore)	Typical use	Protect shared data	Limit resources or coordinate threads

When to Use

Use **mutex** when:

- Protecting shared variables
- Protecting data structures
- Ensuring one thread accesses a critical section

Use **semaphore** when:

- Managing a pool of resources
- Synchronizing producer and consumer
- Coordinating execution between threads

Interview Answer

A mutex is an ownership-based locking mechanism used for mutual exclusion. A semaphore is an integer counter used for synchronization and resource management and does not enforce ownership. Mutexes are ideal for protecting shared data, whereas semaphores are better for coordinating threads or managing multiple resources.

4. Difference: Binary Semaphore vs Mutex ☆☆☆

A **binary semaphore** is a semaphore whose value is restricted to **0** or **1**.

Although it behaves similarly to a mutex, an important difference exists.

Feature	Mutex	Binary Semaphore	Values	Locked / Unlocked	0 or 1	Ownership	Yes	No	Unlock	Only owner	Any thread	Primary use	Mutual exclusion	Signaling and synchronization	Deadlock detection support	Usually supported	Generally not
---------	-------	------------------	--------	-------------------	--------	-----------	-----	----	--------	------------	------------	-------------	------------------	-------------------------------	----------------------------	-------------------	---------------

Key Difference

Mutex:

Thread A locks

↓

Only **Thread A** unlocks

Binary Semaphore:

Thread A waits

↓

Thread B signals

Allowed.

Interview Answer

Both allow only one thread to proceed at a time, but a mutex enforces ownership whereas a binary semaphore does not. Only the thread that acquires a mutex can release it, while any thread may signal a binary semaphore. Therefore, mutexes are preferred for protecting critical sections, whereas binary semaphores are commonly used for synchronization.

5. Counting Semaphore vs Binary Semaphore ☆☆

Binary Semaphore

Possible values

0

1

Allows only one thread/resource.

Example:

```
One printer

Semaphore = 1
```

Counting Semaphore

Possible values

```
0
1
2
...
N
```

Represents multiple identical resources.

Example

```
10 database connections

Semaphore = 10
```

Up to ten threads can access the resource simultaneously.

Comparison

Binary Semaphore	Counting Semaphore	Values: 0 or 1	Values: 0 to N	One resource	Multiple identical resources	Used like a lock or signal
Used for resource pools	Only one thread proceeds	Multiple threads may proceed				

Interview Answer

A binary semaphore has only two possible values, 0 and 1, and is used when only one thread or resource is involved. A counting semaphore maintains a count representing multiple identical resources, allowing several threads to access different instances simultaneously.

6. Spinlock vs Mutex ★★ ★

Mutex

If the lock is unavailable:

```
Thread

↓

Blocked (sleep)

↓

OS wakes later
```

This avoids CPU wastage but incurs a context switch.

Spinlock

If the lock is unavailable:

```
while(lock == busy) {
    // continuously check
}
```

The thread repeatedly checks the lock instead of sleeping.

This is called **busy waiting**.

Comparison

Feature	Mutex	Spinlock	Waiting	Sleeps	Busy waits	Context switch	Yes	No	CPU usage	Low	High	Suitable for	Long waits	Very short
waits	Common	usage	User-space	programs	OS kernel, low-level	code								

When to Use

Use **mutex** if:

- Lock may be held for a relatively long time.
- Context-switch overhead is acceptable.

Use **spinlock** if:

- Lock is expected to be released almost immediately.
- Context switching would cost more than spinning.

Interview Answer

A mutex blocks the waiting thread and allows the operating system to schedule another thread, making it efficient for longer waits. A spinlock continuously checks the lock without sleeping, avoiding context switches but consuming CPU cycles. Spinlocks are suitable only for very short critical sections.

7. Busy Waiting ★ ★

Definition

Busy waiting is a waiting technique in which a thread repeatedly checks a condition instead of sleeping.

Example:

```
while(lock == busy) {  
    // keep checking  
}
```

The thread consumes CPU cycles even though it is doing no useful work.

Characteristics

- Continuously polls a condition
- No context switch
- High CPU utilization
- Very low waiting latency

Advantages

- Extremely fast for short waits
- Eliminates context-switch overhead

Disadvantages

- Wastes CPU cycles
- Poor scalability
- Inefficient for long waiting periods

Where is Busy Waiting Used?

- Spinlocks
- Low-level kernel synchronization
- Multiprocessor systems where locks are held briefly

Interview Answer

Busy waiting is a synchronization technique where a thread continuously checks a condition until it becomes true instead of blocking. It avoids context switches and offers low latency but wastes CPU cycles, so it is suitable only when the expected waiting time is very short.

Quick Revision Table

PrimitiveOwnershipSleepsBusy WaitSupports Multiple ResourcesTypical Use	Mutex	✓	Yes	✓	Yes	✗	No	✗	No	Protect critical sections								
Binary Semaphore	✗	No	✓	Yes	✗	No	✗	No	Signaling or mutual exclusion	Counting Semaphore	✗	No	✓	Yes	✗	No	✓	Yes
Resource pools	Spinlock	✓	Yes	✗	No	✓	Yes	✗	No	Very short critical sections								

Common Interview One-Liners

- **Mutex:** "Ownership-based lock for mutual exclusion."
- **Semaphore:** "Counter-based synchronization primitive using `wait()` and `signal()`."
- **Binary Semaphore:** "Semaphore with values 0/1; no ownership."
- **Counting Semaphore:** "Controls access to multiple identical resources."
- **Spinlock:** "Busy-waiting lock that avoids context switches."
- **Busy Waiting:** "Continuously polling a condition while consuming CPU cycles."
